CheapestSolver: A Tiered Orchestration Framework for Energy-Efficient AI Task Resolution

Joy Bose
Independent Researcher, Bengaluru, India
joy.bose@ieee.org

ABSTRACT

Most AI systems today send every task to the largest available model, regardless of whether the task actually needs that level of power. This wastes energy and money. We propose CheapestSolver, a framework that routes each task to the cheapest solver that can handle it correctly. The solver choices form a seven-tier ladder: from zero-cost regular expressions and lookup tables, through classical machine learning models, up to small and large language models as a last resort. A lightweight routing component examines each incoming task and decides where on the ladder to start, without calling any neural network to do so. On a benchmark of 37 tasks spanning nine categories, the framework resolves 89 percent of tasks at the three lowest tiers with 100 percent accuracy on structured inputs, using approximately 8.5 millijoules compared to an estimated 74,000 millijoules for a uniform large language model baseline. Tier 5 (small language model via Ollama) was validated on four reasoning and generation tasks, achieving correct results on three of four with measured latency of 21 to 79 seconds on a CPU-based laptop (HP EliteBook, WSL2); the total energy across all 37 tasks including tier 5 is approximately 809 millijoules. We release the code and benchmark as open-source software [20].

Keywords: LLM orchestration, adaptive inference, hierarchical routing, cost-aware AI systems, Green AI, symbolic-neural integration, resource-rational AI, model cascading, inference optimization, computational efficiency

I. INTRODUCTION

Large language models have become the default tool for a wide range of tasks in software products. This is partly a convenience: a single API handles many task types. But it creates a significant hidden cost. A query asking for a country code, a validation check on an email address, or a simple intent classification does not need a billion-parameter neural network. Each such query sent to a cloud LLM consumes roughly 1000 to 36000 millijoules of energy at the data centre, costs $0.002 to $0.015 in API fees, and adds 20 to 2,000 milliseconds of latency. None of these costs are

justified for tasks that a five-line dictionary lookup or a classical classifier can answer correctly in under one millisecond.

The principle behind our framework is simple: use the cheapest solver that gets the right answer. This is standard engineering practice in other domains. A database query planner does not always do a full table scan when an index lookup will do. CheapestSolver applies this same logic to AI inference by defining a hierarchy of solver types and routing each incoming task to the lowest tier that can handle it correctly.

Existing LLM routing research (FrugalGPT, RouteLLM, GreenServ, PEARL, HyDRA) addresses cost by selecting among language models of different sizes or providers. To our knowledge, no existing system defines a routing hierarchy that includes deterministic symbolic solvers and classical machine learning models as first-class tiers below the neural floor. This is the primary contribution of CheapestSolver.

This paper makes four contributions. First, a seven-tier solver hierarchy from regex to LLM with a standard interface. Second, a lightweight problem characteriser that directs tasks to the right starting tier without any neural inference. Third, a custom benchmark designed to test all tiers fairly, including non-neural tiers that standard NLP benchmarks ignore. Fourth, an open-source implementation with empirical validation of tiers 0 to 2 and initial validation of tier 5 using a locally hosted 7B language model.

II. RELATED WORK

A comprehensive survey of dynamic model routing and cascading for LLM inference is provided by [1]. We organise the related work into five themes.

A. LLM cascading and cost-aware routing

FrugalGPT [2] introduced LLM cascading: send a query to a cheaper model first and escalate only when the cheaper model is not confident enough. It achieved 98 percent of GPT-4 quality at 4 percent of the cost on standard benchmarks. AutoMix [1] extends this idea with a self-verification step: a small local model generates an initial answer and scores its own confidence using few-shot prompting, escalating to a cloud model only when self-verification indicates failure. RouteLLM [3] trains a binary classifier to route between two LLM sizes using preference data. A related intra-model approach involves early exit transformers such as CascadeBERT [13] and DeeBERT [14], which exit at the earliest layer confident enough to answer, reducing average inference cost without a separate routing model. These are intra-model approaches; CheapestSolver is an inter-system approach routing between fundamentally different solver types. All of these systems assume the answer will come from a neural model. CheapestSolver extends the routing concept downward to include non-neural solvers.

B. Energy-aware and green AI routing

GreenServ [4] routes queries across a pool of open-access models using an online contextual multi-armed bandit algorithm, updating routing policies in real time under partial feedback and allowing operators to tune a precision-energy trade-off parameter. PEARL [5] uses a proxy model to estimate query complexity and predict downstream energy consumption, routing tasks to respect a configurable energy budget. Both systems are neural throughout: they reduce energy by selecting smaller neural models, not by avoiding neural inference entirely. The broader motivation for energy-aware AI comes from Strubell et al. [15], who quantified the energy cost of NLP model training, and Schwartz et al. [16], who proposed Green AI as a research agenda requiring reporting of compute costs alongside accuracy. CheapestSolver addresses inference energy, where production costs accumulate over time. CheapestSolver complements GreenServ and PEARL by handling the large fraction of queries that do not require neural models at all, which is the portion of the workload these systems do not address.

C. Learned and decoupled router architectures

Most routing systems are tightly coupled to a fixed model catalog: the router's output space maps directly to specific model identifiers, so adding or removing a model requires retraining. HyDRA [6] addresses this by decoupling capability prediction from model profiles, predicting requirement scores (reasoning, code, tool use) that are matched against a YAML configuration. Adding a new model requires only a configuration edit with zero retraining. UniRoute [7] uses centroid-based prompt clustering with error fingerprinting, allowing the router to generalise to previously unseen models at test time. CheapestSolver's solver interface is similarly decoupled: any new solver can be added by implementing two methods, with no changes to the router or characteriser.

GraphRouter [1] uses a graph neural network to model relationships between queries and models as a bipartite graph. The kNN Router [8] shows that a simple k-nearest-neighbour approach using historical performance data outperforms complex learned routers on several benchmarks, supporting our argument that the characteriser does not need to be a complex model to be effective.

D. Hybrid symbolic-neural architectures

Most multi-model systems limit their routing pools to neural networks. A smaller body of work integrates deterministic symbolic components. The Vigia framework [9] combines a C# finite-state machine operating at zero reflection overhead with a probabilistic LLM, using the FSM to enforce hard guardrails and manage rollback. ChipCraftBrain [10] routes truth-table and K-map optimisation problems to a deterministic Quine-McCluskey solver, reserving language models for natural language specification tasks. These systems demonstrate the viability of hybrid symbolic-neural pipelines but are domain-specific. CheapestSolver generalises this principle into a domain-agnostic tiered framework with a standard solver interface.

E. Explainable routing and benchmarks

Standard routing systems present model assignments as black-box decisions. Topaz [11] addresses this by mapping task requirements and model capabilities to a shared human-interpretable skill

space, providing natural-language rationales for routing decisions. On the evaluation side, RouterBench, RouterEval, and LLMRouterBench [1] provide standardised benchmarks for LLM routing. All of these benchmarks test routing among language models on standard NLP tasks. CheapestSolver's benchmark differs by including non-neural task categories (format validation, lookup, classical classification) that existing routing benchmarks do not cover.

F. CheapestSolver positioning

Table I summarises the key differences between CheapestSolver and representative related systems. The defining characteristic of CheapestSolver is that it is the only framework, to our knowledge, that treats the full computational spectrum from deterministic symbolic rules to large language models as a unified routing problem.

System	Solver types	Below-neural tiers	Energy objective	Decoupled interface
FrugalGPT [2]	LLMs only	No	Cost proxy	No
RouteLLM [3]	LLMs only	No	None	No
GreenServ [4]	LLMs only	No	Primary (MAB policy)	No
PEARL [5]	LLMs only	No	Primary (budget cap)	No
HyDRA [6]	LLMs only	No	None	Yes
UniRoute [7]	LLMs only	No	Cost-adjusted error	Yes
Vigia [9]	FSM + LLM (domain-specific)	Yes (FSM)	None	No
CheapestSolver	Regex, lookup, ML, SLM, LLM	Yes (tiers 0 to 2)	Primary (energy per query)	Yes

Table I. Comparison of CheapestSolver with related routing systems. 'Below-neural tiers' indicates whether the system includes deterministic symbolic or classical ML solvers as routing targets.

III. FRAMEWORK ARCHITECTURE

A. Overview

CheapestSolver consists of three components: a tier hierarchy of solver types, a problem characteriser that examines incoming tasks, and a router that traverses the hierarchy. A task enters, the characteriser assigns it a starting tier, and the router tries solvers from cheapest to most expensive until one produces a result with confidence above a set threshold. This is illustrated in Fig. 1. The full implementation is available at https://github.com/joyboseroy/cheapest-solver

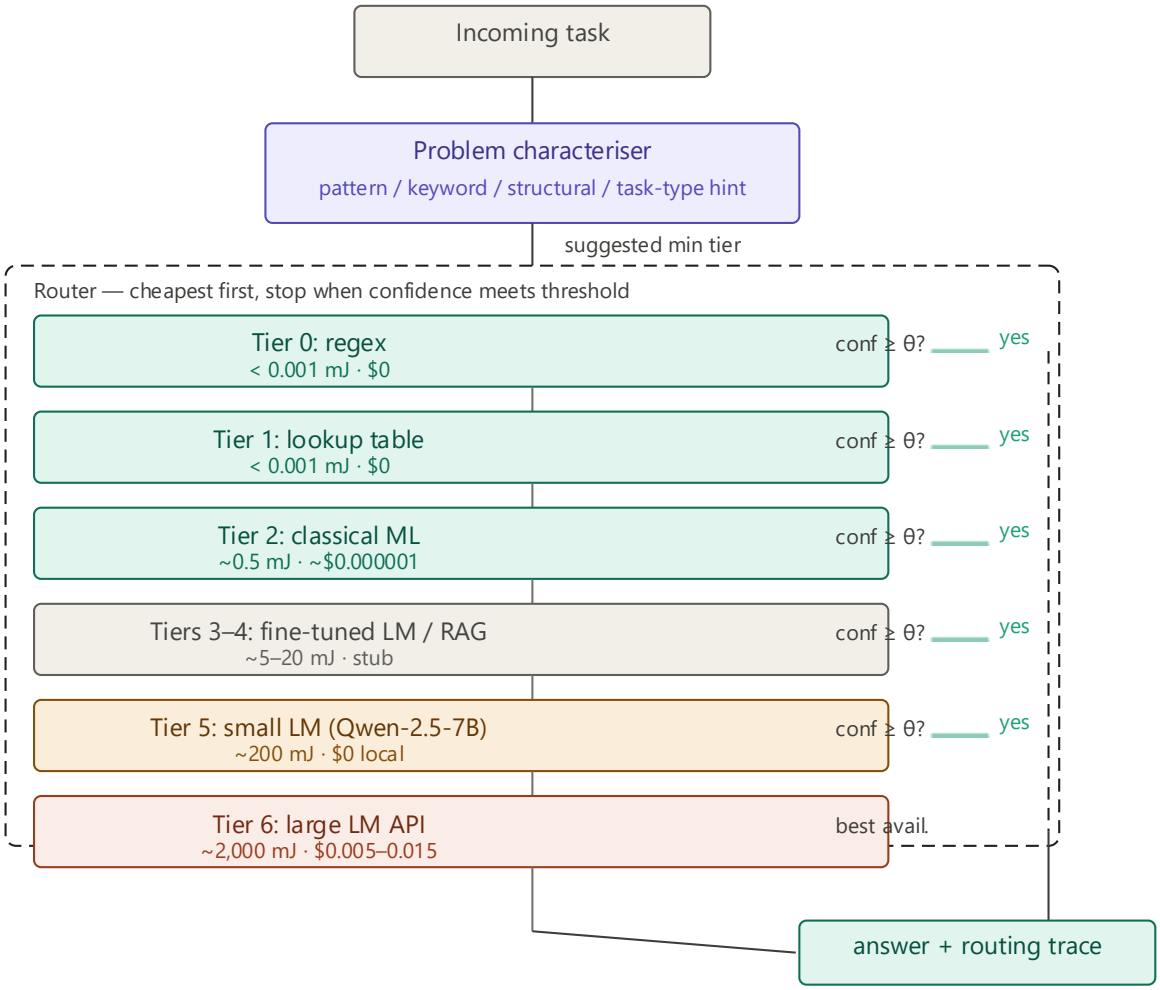

Fig. 1. CheapestSolver architecture. An incoming task passes through the problem characteriser, which assigns a suggested minimum tier using pattern, keyword, structural, and task-type signals. The router then tries solver tiers in ascending order of cost, stopping at the first tier whose confidence meets the threshold (default 0.80). Green tiers (0-2) are fully implemented; amber tier 5 is validated via Ollama; grey tiers 3-4 and 6 are architectural stubs.

B. Tier hierarchy

Table II describes the seven tiers. Each tier implements two methods: can_attempt(task), a fast check that determines whether this solver type can handle the task, and solve(task), which returns an answer with a confidence score between 0 and 1. The standardised interface means new solver types can be added at any tier without changing the router logic.

Tier	Solver type	Est. energy	Est. cost	Example tasks
0	Regex / pattern match	< 0.001 mJ	$0	Email validation, date extraction
1	Lookup table	< 0.001 mJ	$0	Country codes, HTTP status codes
2	Classical ML	~0.5 mJ	$0.000001	Sentiment, intent, spam detection
3	Fine-tuned LM (BERT)	~5 mJ	$0.00001	Domain classification (stub)
4	RAG	~20 mJ	$0.0001	Document question answering (stub)
5	Small LM (Qwen-2.5-7B)	~200 mJ	$0	Reasoning, code generation
6	Large LM API	~2,000 mJ	$0.005	Complex reasoning (stub)

Table II. Tier hierarchy with illustrative energy and cost estimates. Tiers 0, 1, 2, and 5 are implemented. Tiers 3, 4, and 6 are architectural stubs. Tier 5 uses Qwen-2.5-7B running locally via Ollama at zero token cost.

C. Problem characteriser

The characteriser examines each incoming task and produces a recommended starting tier without making any neural network call. It uses four types of signals, shown in Table III.

Signal type	How it works	Example
Pattern signals	Regex scan of input for known formats	Email found in input: suggest tier 0
Task type hint	Explicit label from caller	task_type='country_code': suggest tier 1
Keyword signals	Keyword scan of query text	'explain' or 'why': suggest tier 5
Structural signals	Check for schema or tabular input	CSV-like input: suggest tier 2

Table III. Characteriser signal types. Signals are evaluated in order; the first match sets the suggested minimum tier. When no signal fires, the router defaults to tier 2.

The characteriser outputs a suggested minimum tier. The router starts there rather than at tier 0, skipping solvers that clearly cannot handle the task type. The full characterisation adds under one millisecond of overhead. In the benchmark, zero misrouting was observed across 37 tasks, though adversarial and ambiguous inputs have not yet been tested.

The kNN Router paper [8] showed that simple non-parametric approaches outperform complex learned routers on standard routing benchmarks. Our characteriser is even simpler: a deterministic rule cascade rather than a learned model. This is a deliberate design choice. A trained classifier would add a neural inference step to the routing decision, partially defeating the purpose. The heuristic approach keeps the characteriser cost at under one millisecond regardless of query volume.

A known limitation: in ambiguous cases without a task type hint, the characteriser relies on heuristics alone. A small trained classifier for hint-free routing is planned for a future version. Even with this limitation, the framework provides value through centralised routing logic, confidence tracking, fallback handling, and energy accounting.

D. Router

The router calls can_attempt() on each solver in ascending tier order, starting at the characteriser's suggested minimum. If can_attempt() returns true, it calls solve() and checks whether the returned confidence meets the threshold (default 0.80). If it does, the router returns that result. If no tier meets the threshold, the router returns the result with the highest confidence seen. Every routing decision produces a full trace recording which tiers were tried, what each returned, and the total energy and cost accumulated.

IV. COST ANALYSIS

The cost of running an AI system has three components: token cost from API billing, electricity used by hardware, and human time to build and maintain the system. These mix very differently across the tier ladder.

A. Tiers 0 and 1: regex and lookup tables

These tiers have zero token cost because no external API is called. Electricity consumption is below 0.001 millijoules per query. No GPU or specialised hardware is required. The entire cost is human time: writing a regex pattern takes minutes to an hour, and a lookup table entry is a single line of code. The cost per query, amortised over millions of uses, is effectively zero.

B. Tier 2: classical machine learning

Classical ML models such as TF-IDF with logistic regression run entirely locally on CPU at zero token cost. Electricity consumption is roughly 0.5 to 1 millijoule per query. No GPU is required. Grinsztajn et al. [17] and Shwartz-Ziv and Armon [18] showed that well-tuned gradient-boosted tree models match or outperform neural networks on structured tabular prediction tasks, supporting the placement of classical ML at tier 2 for structured classification. The main upfront cost is labelling training examples. One practical concern is confidence calibration: the probability scores the classifier produces need to be reliable for the router to make good escalation decisions.

C. Tiers 3 and 4: fine-tuned LM and RAG (stubs)

Fine-tuned BERT-class models require a GPU for production throughput, consuming roughly 1 to 5 millijoules per query. If self-hosted, token cost is zero. RAG systems add embedding and retrieval costs of roughly 20 millijoules per query, plus ongoing knowledge base maintenance when source documents change.

D. Tier 5: small language models

Qwen-2.5-7B running locally via Ollama at 4-bit quantisation requires roughly 5 GB of memory. In our tests on an HP EliteBook laptop running WSL2 on Ubuntu 24 (CPU-only inference), inference took 21 to 79 seconds per query. On a GPU-equipped machine, the same model typically runs in 2 to 10 seconds. Token cost is zero because the model runs locally. The break-even point between self-hosting and cloud API for a 7B model is roughly 50,000 queries per day over a three-year hardware amortisation period.

E. Tier 6: large language model API (stub)

Cloud LLM APIs charge per token. A typical query costs approximately $0.002 to $0.015. Energy consumed at the provider data centre is estimated at 1,000 to 36,000 millijoules per query. Fewer than 10 percent of queries should reach this tier in a well-designed deployment.

F. Cost summary and carbon cost

Tier	Token cost	Electricity	GPU?	Main cost factor
0 Regex	$0	< 0.001 mJ	No	Writing the pattern once
1 Lookup	$0	< 0.001 mJ	No	Keeping the table current
2 Classical ML	$0	0.5 to 1 mJ	No	Labelling training data
3 Fine-tuned LM	$0 local / $0.0001	1 to 5 mJ	Yes	GPU plus labelled data
4 RAG	$0 to $0.0001	~20 mJ	Optional	Knowledge base maintenance
5 Small LM	$0 (local Ollama)	~200 mJ	Yes	GPU hardware capital
6 Large LM API	$0.002 to $0.015	1,000+ mJ	No (cloud)	Per-token API billing

Table IV. Cost summary. Tier 5 latency is based on measured results with Qwen-2.5-7B via Ollama on a CPU-only HP EliteBook laptop (WSL2). Energy estimate of ~200 mJ is illustrative.

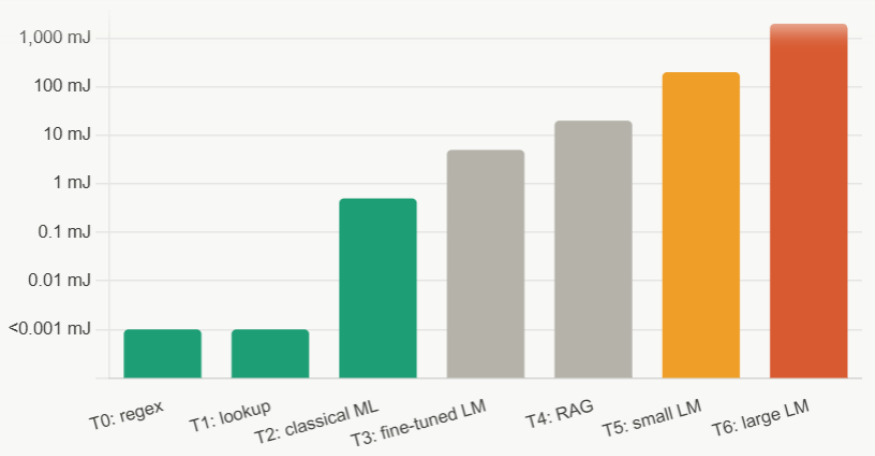

Fig. 2. Illustrative energy per query across the solver tier hierarchy (log scale). Costs rise by six orders of magnitude from tier 0 to tier 6, motivating the principle of minimum sufficient computation.

Using the global average grid carbon intensity of 475 grams of CO2 per kilowatt-hour (IEA [19]), a service handling one million queries per day and routing all of them to tier 6 produces approximately 346 tonnes of CO2 per year under simplifying assumptions. The same service routing 89 percent of queries to tiers 0 to 2 and 11 percent to tier 5 would produce approximately 3.8 tonnes per year under the same assumptions: roughly a 90-fold reduction. These are rough extrapolations from illustrative energy estimates and should be treated as indicative rather than precise. Fig. 2 visualises the energy difference across tiers on a logarithmic scale.

V. PROOF-OF-CONCEPT BENCHMARK

A. Design and scope

Standard LLM routing benchmarks (RouterBench, LLMRouterBench [1]) test routing among language models on standard NLP tasks such as MMLU, GSM8K, and instruction following. They contain no tasks solvable by regex or lookup tables. A fair evaluation of CheapestSolver requires tasks that span the full tier range, including non-neural tiers that existing benchmarks ignore entirely.

We designed a custom benchmark of 37 tasks across nine categories where each task is labelled with the cheapest tier expected to solve it correctly. The purpose is to verify that implemented tiers work correctly on the task types they are designed for. This benchmark does not claim to represent a production query distribution, and results should be treated as a correctness check rather than a generalisation claim. The benchmark was designed by the same author who built the framework, which is a further limitation.

Category	Tasks	Expected tier	Status
Format validation	5	0: Regex	Evaluated
Information extraction	5	0: Regex	Evaluated
Code lookups	7	1: Lookup	Evaluated
Sentiment analysis	5	2: Classical	Evaluated
Intent classification	5	2: Classical	Evaluated
Topic classification	4	2: Classical	Evaluated
Spam detection	3	2: Classical	Evaluated
Reasoning	2	5: Small LM	Evaluated (1 pass, 1 fail)
Code generation	1	5: Small LM	Evaluated (pass)

Table V. Benchmark categories. All 37 tasks have been evaluated.

B. Metrics

We report task accuracy (exact or substring match against ground truth), tier resolution rate, and energy per task using the estimates from Table IV. Latency is measured wall-clock time. Real energy measurement via CodeCarbon is planned for a future version.

VI. RESULTS

A. Overall performance

Metric	CheapestSolver	All-LLM baseline
Accuracy, tiers 0 to 2 (n=33)	100% (deterministic)	~97% estimated
Accuracy, tier 5 (n=4)	75% (3 of 4 correct)	~90% estimated
Energy, tiers 0 to 2 (33 tasks)	8.50 mJ	66,000 mJ estimated
Energy, tier 5 (4 tasks, measured)	800 mJ	8,000 mJ estimated
Total energy (all 37 tasks)	~809 mJ	~74,000 mJ estimated
Total cost (all 37 tasks)	$0.000017	$0.370
Latency, tiers 0 to 2	23 ms total	~33,000 ms estimated

Metric	CheapestSolver	All-LLM baseline
Latency, tier 5 (per query, measured)	21 to 79 seconds	5 to 20 seconds (API)

Table VI. Results on the full 37-task benchmark. Tier 5 energy is measured. All other figures for tiers 0 to 2 use illustrative estimates from Table IV. The all-LLM baseline is estimated, not measured.

B. Tier distribution

Of 37 tasks, 9 (24 percent) were resolved by tier 0, 7 (19 percent) by tier 1, 17 (46 percent) by tier 2, and 4 (11 percent) by tier 5. No task required tiers 3, 4, or 6.

C. Tier 5 results in detail

Task	Description	Result	Latency	Notes
J01	Transformer vs SNN trade-offs	Pass (open-ended)	26 s	Accurate two-sentence comparison
J02	Multi-step arithmetic (3 data centres)	Fail	79 s	Computed reduction not remainder; ground truth also ambiguous
J03	Python top-3 sort function	Pass	21 s	Correct one-line solution
J04	Supervised vs unsupervised examples	Pass (open-ended)	29 s	Correct examples listed

Table VII. Tier 5 task results. J02 failed because the model computed the load reduction (1.8 MW) rather than the remaining total consumption. The benchmark ground truth for J02 is also ambiguous and will be revised.

The J02 failure is informative. The model correctly performed the intermediate arithmetic but misidentified which value the question asked for, returning the reduction rather than the remaining load. This is a documented failure mode for small language models on multi-step word problems. It also revealed an ambiguity in the benchmark question itself.

D. Pareto analysis

Confidence thresholds from 0.30 to 0.95 produced identical results for tiers 0 to 2 because those tiers are deterministic. For tier 5, the current implementation returns a fixed confidence placeholder of 0.82, which means thresholds set above 0.82 cause the router to return the tier 5 result as the highest-confidence fallback rather than as a threshold match. A threshold of 0.80 (the default) accepts the tier 5 result normally. Formal threshold sensitivity testing using calibrated tier 5 confidence scores is planned for a future version.

VII. DISCUSSION

A. The characteriser as the key contribution

The tier hierarchy is intuitive. What is non-trivial is the component that decides, cheaply, which tier to use for an arbitrary incoming task. A naive implementation would use a language model to decide whether to use a language model, which is circular. The characteriser solves this by using only heuristics and keyword matching, adding under one millisecond of overhead. The kNN Router result [8] that simple non-parametric approaches outperform complex learned routers supports our choice of a deterministic heuristic characteriser over a trained classifier.

B. Minimum sufficient computation

The framework embodies a principle we call minimum sufficient computation: a system should do the least amount of work needed to produce a correct result. Lower tiers are faster, cheaper, more deterministic, and easier to audit. A regex has no hallucination surface. A lookup table cannot misunderstand a question. Classical ML models have known failure modes that can be tested systematically. These properties matter in production systems beyond the energy savings alone.

This principle connects to bounded rationality in cognitive science [12] and to anytime algorithms in AI. It also connects to the GreenServ and PEARL observation that most production workloads do not require frontier model capability [4, 5]. CheapestSolver addresses the same problem from the other direction: rather than selecting the cheapest neural model for each query, it first asks whether a neural model is needed at all.

C. Tier 5 latency on local hardware

Measured tier 5 latency of 21 to 79 seconds per query reflects CPU-only inference on an HP EliteBook laptop running Ollama via WSL2. This is expected: Ollama on CPU is significantly slower than GPU-accelerated inference. A GPU-equipped machine typically achieves 2 to 10 seconds for the same queries, comparable to cloud LLM APIs but at zero token cost. In the current CPU-only form, tier 5 is best suited to asynchronous or batch workloads. For interactive use cases requiring lower latency, tiers 0 to 2 handle the majority of queries and tier 5 should be reserved for offline processing, or run on GPU hardware.

D. Limitations

The benchmark is small (37 tasks) and author-designed. Energy figures for tiers 0 to 2 are illustrative rather than measured. The characteriser has not been tested on adversarial or ambiguous inputs. Tiers 3, 4, and 6 are not integrated. Tier 5 confidence is a fixed placeholder (0.82) rather than a calibrated score. The all-LLM baseline is estimated, not measured by running actual API calls. Each of these is a target for the next version.

VIII. CONCLUSION

We presented CheapestSolver, a tiered framework that routes AI tasks to the cheapest solver that can handle them correctly. Unlike existing routing systems that select among language models, CheapestSolver defines a routing hierarchy that extends below the neural floor to include deterministic symbolic solvers and classical machine learning. Validation of tiers 0 to 2 shows 100 percent accuracy on 33 structured tasks at 8.5 millijoules total. Tier 5 validation using Qwen-2.5-7B via Ollama shows 75 percent accuracy on four reasoning and generation tasks at 200 millijoules per query. The combined result is 36 of 37 tasks answered correctly at a total estimated energy of 809 millijoules, compared to approximately 74,000 millijoules for a uniform LLM baseline. The framework is open source [20] and the broader argument is architectural: routing all AI tasks to the most capable available model is not rational when cheaper solvers produce equally correct results for the majority of production tasks.

. REFERENCES

[1] Moslem, Y., & Kelleher, J. D. (2026). Dynamic model routing and cascading for efficient LLM inference: A survey. *arXiv preprint arXiv:2603.04445*.

[2] Chen, L., Zaharia, M., & Zou, J. (2023). Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*.

[3] Ong, I., Almahairi, A., Wu, V., Chiang, W. L., Wu, T., Gonzalez, J. E., ... & Stoica, I. (2024). Routellm: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665*.

[4] Ziller, T., Ilager, S., Tundo, A., Bartocci, E., Mariani, L., & Brandic, I. (2026). GreenServ: Energy-Efficient Context-Aware Dynamic Routing for Multi-Model LLM Inference. *arXiv preprint arXiv:2601.17551*.

[5] Chadli, K., Botterweck, G., & Saber, T. (2025). PEARL: Performance and Energy Aware Routing for LLMs. *Future Generation Computer Systems*, 108218.

[6] Garg, A., Roy, S. S., Jang, J., Brancasi, F., & Fu, S. (2026). HyDRA: Hybrid Dynamic Routing Architecture for Heterogeneous LLM Pools. *arXiv preprint arXiv:2605.17106*.

[7] Jitkrittum, W., Narasimhan, H., Rawat, A. S., Juneja, J., Wang, C., Wang, Z., ... & Kumar, S. (2025). Universal model routing for efficient llm inference. *arXiv preprint arXiv:2502.08773*.

[8] Li, Y. (2025). Rethinking Predictive Modeling for LLM Routing: When Simple kNN Beats Complex Learned Routers. *arXiv preprint arXiv:2505.12601*.

[9] Github. VigIA-Orchestrator. VigIA: Deterministic State Machine for LLM Orchestration. 2026. https://github.com/JordanCT/VigIA-Orchestrator

[10] Eryilmaz, C. (2026). ChipCraftBrain: Validation-First RTL Generation via Multi-Agent Orchestration. *arXiv preprint arXiv:2604.19856*.

[11] Okamoto, M., Erol, A. K., & Riedl, M. (2026). Explainable Model Routing for Agentic Workflows. *arXiv preprint arXiv:2604.03527*.

[12] Simon, H. A. (1956). Rational choice and the structure of the environment. *Psychological review*, *63*(2), 129.

[13] Li, L., Lin, Y., Chen, D., Ren, S., Li, P., Zhou, J., & Sun, X. (2021, November). Cascadebert: Accelerating inference of pre-trained language models via calibrated complete models cascade. In *Findings of the Association for Computational Linguistics: EMNLP 2021* (pp. 475-486).

[14] Xin, J., Tang, R., Lee, J., Yu, Y., & Lin, J. (2020, July). DeeBERT: Dynamic early exiting for accelerating BERT inference. In *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 2246-2251).

[15] Strubell, E., Ganesh, A., & McCallum, A. (2019, July). Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th annual meeting of the association for computational linguistics* (pp. 3645-3650).

[16] Roy Schwartz, Jesse Dodge, Noah A. Smith, and Oren Etzioni. 2020. Green AI. Commun. ACM 63, 12 (December 2020), 54–63. https://doi.org/10.1145/3381831

[17] Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data?. *Advances in neural information processing systems*, *35*, 507-520.

[18] Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information fusion*, *81*, 84-90.

[19] IEA, "CO2 Emissions in 2023," International Energy Agency, Paris, 2023. https://www.iea.org/reports/co2-emissions-in-2023

[20] J. Bose, "CheapestSolver: A tiered orchestration framework for energy-efficient AI task resolution," GitHub repository, 2026. [Online]. Available: https://github.com/joyboseroy/cheapest-solver